

PORTADA

# Calidad de Software

**Semana 11 — Métricas de calidad: densidad de defectos, DRE, cobertura y complejidad ciclomática**

Universidad de Antioquia · Ingeniería de Sistemas



ENCUADRE

## Hoy se entrega el avance 2 de Fábrica-Escuela

Esta semana corresponde la **entrega 2 de Fábrica-Escuela** (10 % de la nota), con los temas propios del proyecto de cada equipo — independiente del taller de seguimiento de hoy.

Seguimos en la **Unidad 3 — Implementación del proceso de aseguramiento**, con los equipos fijos.



REPASO RELÁMPAGO

## Lo que ya tenemos de la semana 10

- Una prueba unitaria es **rápida, aislada, determinista y enfocada**.
- **Arrange-Act-Assert** ordena cualquier prueba en tres pasos.
- Un **test double** reemplaza una dependencia real: stub, mock o fake.



TRANSICIÓN

## Hoy: medir la calidad con números

Hasta ahora hemos hablado de calidad con palabras: características, tipos, técnicas. Hoy se traduce parte de eso a **números** — indicadores que permiten comparar, hacer seguimiento en el tiempo, y tomar decisiones basadas en evidencia en vez de impresiones.



## CONCEPTO

# Qué es una métrica de software

Es una **medida cuantitativa** del grado en que un sistema, componente o proceso posee un atributo dado. No es lo mismo que una simple medición: la métrica típicamente **combina** mediciones —por ejemplo, defectos y líneas de código— en un indicador con significado propio.

Hoy vemos cuatro métricas clásicas de calidad de software: densidad de defectos, DRE, cobertura y complejidad ciclomática.



## CONCEPTO

# Por qué medir

Una métrica sirve para **tomar decisiones**: dónde concentrar el esfuerzo de pruebas, si un módulo necesita refactorización, si el equipo está mejorando de un sprint a otro.

**Ninguna métrica de este curso se sostiene sola.** Cada una revela una parte de la historia y oculta otra — por eso el taller de hoy no pide solo calcular números, sino **interpretarlos** con sentido crítico.



TRANSICIÓN

## Empezamos por los defectos

La primera métrica responde la pregunta más directa: de todo lo que se construyó, ¿cuánto tiene defectos?



## CONCEPTO

# Densidad de defectos

Mide **cuántos defectos hay por unidad de tamaño** del software — normalmente por cada mil líneas de código (KLOC):

Densidad de defectos = Número de defectos ÷ Tamaño (en KLOC)

**Ejemplo:** un módulo de 4.000 líneas con 12 defectos reportados tiene una densidad de **3 defectos por KLOC**. Comparar esa cifra contra otros módulos —o contra el mismo módulo a lo largo del tiempo— es lo que la vuelve útil.



## CONCEPTO

# Qué revela y qué oculta la densidad de defectos

|        |                                                                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revela | Qué módulos concentran más defectos en relación con su tamaño — útil para priorizar dónde revisar o refactorizar.                                                                                           |
| Oculta | La <b>severidad</b> de esos defectos (diez defectos triviales pesan igual que diez críticos en la fórmula), y si el conteo de líneas de código es siquiera una buena medida de "tamaño" o complejidad real. |

Un módulo con baja densidad no es necesariamente un módulo de calidad — puede ser un módulo poco probado, con pocos defectos encontrados todavía.

TRANSICIÓN

## Qué tan buena fue la red de pruebas

La densidad de defectos cuenta cuántos hay. La siguiente métrica responde algo distinto: de los defectos que existían, ¿cuántos atrapó el proceso de pruebas **antes** de que el usuario los encontrara?



## CONCEPTO

# DRE — Eficiencia de eliminación de defectos

**DRE** (*Defect Removal Efficiency*) mide qué porcentaje de los defectos totales se encontró **antes** de que el software llegara a producción:

$$\text{DRE} = \text{Defectos encontrados antes del release} \div (\text{Defectos antes} + \text{Defectos encontrados después}) \times 100$$

Es, en números, la misma idea detrás de los criterios de salida de un plan de pruebas (semana 6): cuánta confianza da el proceso de pruebas antes de soltar el software al mundo.



EJEMPLO

## Calculando el DRE

Un equipo encontró **45 defectos** durante sus pruebas antes del release. En el primer mes en producción, los usuarios reportaron **5 defectos** más.

$$\text{DRE} = 45 \div (45 + 5) \times 100 = \mathbf{90 \%}$$

El 90 % de los defectos se atraparon antes de llegar al usuario. Un DRE del 100 % significaría que ningún defecto escapó a producción — en la práctica, casi imposible de sostener siempre.

CONCEPTO

# Qué revela y qué oculta el DRE

|        |                                                                                                                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revela | Qué tan efectivo es el proceso de pruebas completo para atrapar problemas antes del release.                                                                                                                         |
| Oculto | Solo puede calcularse <b>después</b> de que pasó un tiempo en producción — no sirve como alerta temprana. Y un DRE alto no dice nada sobre defectos que <b>nadie</b> —ni pruebas ni usuarios— ha encontrado todavía. |

TRANSICIÓN

## Cobertura, otra vez

Ya se vio en la semana 5 (con SonarCloud) y en la semana 8 (sentencia, decisión, camino): qué porcentaje del código se ejecutó durante las pruebas. Hoy entra formalmente al catálogo de métricas del curso, con la misma advertencia de siempre.



CONCEPTO · RECORDATORIO

# Qué revela y qué oculta la cobertura

|        |                                                                                                                                                                                                |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revela | Qué porcentaje del código se ejecutó al menos una vez durante las pruebas — útil para encontrar zonas completamente sin probar.                                                                |
| Oculta | Si el resultado de esa ejecución realmente se <b>verificó</b> con una aserción. Un 100 % de cobertura con pruebas sin aserciones significativas no dice nada sobre la calidad real del código. |

Es la métrica más fácil de "inflar" artificialmente sin ganar calidad real — tema al que se vuelve más adelante en esta clase.

TRANSICIÓN

## Qué tan enredado está el código por dentro

Las tres métricas anteriores miran el código desde afuera —defectos, releases, ejecución—. La última mira su estructura interna: qué tan difícil es de seguir, probar y modificar.



CONCEPTO · ORIGEN

## Complejidad ciclomática

Propuesta por **Thomas McCabe** en 1976, mide el **número de caminos independientes** que puede recorrer un método o función — en la práctica, se calcula sumando 1 al número de puntos de decisión ( `if` , `while` , `for` , `case` ):

Complejidad = Número de decisiones + 1

Un método sin ningún `if` tiene complejidad 1: solo hay un camino posible. Cada decisión nueva multiplica las rutas que una suite de pruebas necesitaría cubrir para llegar a cobertura de camino completa (semana 8).

EJEMPLO

## Calculando la complejidad ciclomática

```
def clasificar_cliente(compras, antigüedad):  
    if compras > 1000:                # decisión 1  
        if antigüedad > 2:           # decisión 2  
            return "VIP"  
        return "Frecuente"  
    elif compras > 100:               # decisión 3  
        return "Regular"  
    return "Nuevo"
```

**3 decisiones + 1 = complejidad ciclomática 4.** Hacen falta al menos 4 casos de prueba distintos para ejercitar cada camino posible de esta función.

## CONCEPTO

# Qué revela y qué oculta la complejidad ciclométrica

|        |                                                                                                                                                                                                        |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revela | Qué tan difícil es de probar exhaustivamente un método, y —de forma indirecta— qué tan difícil es de mantener sin introducir defectos.                                                                 |
| Oculta | Si el código hace lo <b>correcto</b> . Un método con complejidad 1 puede estar completamente equivocado, y uno con complejidad alta puede ser perfectamente correcto si de verdad necesita esa lógica. |

CONCEPTO · SÍNTESIS

# Las cuatro métricas, lado a lado

| Métrica                 | Mide                                      | Punto ciego principal    |
|-------------------------|-------------------------------------------|--------------------------|
| Densidad de defectos    | Defectos por tamaño                       | No distingue severidad   |
| DRE                     | % de defectos atrapados antes del release | Solo se sabe después     |
| Cobertura               | % de código ejecutado                     | No verifica el resultado |
| Complejidad ciclomática | Caminos posibles en el código             | No dice si es correcto   |

INTERACCIÓN

## Ejercicio rápido: interpretar una métrica

**5 minutos:** un módulo reporta **0 defectos** y **95 % de cobertura**. Un compañero concluye: "es el módulo con mejor calidad del sistema".

¿Qué le falta considerar a esa conclusión? Mencionen al menos dos puntos ciegos de las métricas usadas.

Este ejercicio es el calentamiento del taller de hoy.

CONCEPTO EN DISPUTA

## ¿Se pueden "engañar" las métricas?

La Ley de Goodhart lo resume: "**cuando una medida se convierte en objetivo, deja de ser una buena medida.**" Si a un equipo se le exige 90 % de cobertura, algunos escribirán pruebas triviales sin aserciones reales solo para subir el número. Si se penaliza la complejidad ciclomática alta, algunos dividirán funciones artificialmente sin simplificar nada de verdad.

La postura razonable: las métricas sirven para **guiar preguntas** —¿por qué este módulo tiene tanta complejidad?—, no para automatizar juicios de calidad sin revisión humana.



## SÍNTESIS

### Lo que hay que llevarse de hoy

1. **Densidad de defectos** compara defectos contra tamaño — no distingue severidad.
2. **DRE** mide qué tan bien atrapó el proceso los defectos antes del release.
3. **Cobertura** mide qué se ejecutó, no qué se verificó de verdad.
4. **Complejidad ciclomática** mide caminos posibles, no si el código es correcto.
5. Ninguna métrica reemplaza el juicio humano — **todas revelan algo y ocultan algo más.**



## INSTRUCCIONES

# Taller de hoy: calcular e interpretar métricas

**2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela**

Sobre un repositorio asignado por el docente (pueden usar **SonarCloud** para calcular varias de estas métricas automáticamente), cada equipo debe:

- Calcular **densidad de defectos, cobertura y complejidad ciclomática** del repositorio.
- Escribir una **interpretación** de cada métrica: qué revela sobre este repositorio en particular, y qué información importante deja fuera.

**La entrega cierra al terminar las dos horas.** Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

## Para la próxima semana

**Antes de la clase de la semana 12, hay que llegar con esto leído:**

- Integración continua. Pipeline de calidad: build, test, cobertura, análisis estático.

Seguimos con los equipos fijos de **Fábrica-Escuela**. El taller de la semana 12: pipeline de CI operativo sobre el repositorio de práctica, que ejecute la suite y publique el reporte de cobertura.

